

Software Construction & Fundamentals

- **Software Construction:** The detailed creation of working software through a combination of coding, verification, unit testing, integration testing, and debugging.
- **Construction Planning:** The process of defining and coordinating the tasks, resources, schedule, and procedures to construct software as part of the software development lifecycle.
- **Minimizing Complexity:** Emphasizing code creation that is simple and readable rather than clever.
- **Anticipating Change:** Designing and constructing software so that it can adapt easily when requirements, technology, or environments change.
- **Constructing for Verification:** Building software in such a way that faults can be readily found by the software engineers writing the software as well as by the testers and users.
- **Reuse:** Using existing assets (libraries, modules, components, source code, etc.) in solving different problems.
- **Construction for Reuse:** Creating reusable software assets.
- **Construction with Reuse:** Creating new software with the reuse of existing software assets,.
- **Standards:** Agreed-upon rules, conventions, or guidelines that developers follow when writing and organizing code.
- **Configuration Languages:** Languages primarily used for configuring software and systems (e.g., YAML, JSON).
- **Toolkit Languages:** Languages used to build applications by assembling predefined, reusable components from a toolkit (e.g., LabVIEW).
- **Scripting Languages:** Application programming languages that support scripts which are often interpreted rather than compiled.
- **Linguistic Programming Languages:** Languages that use text-based syntax with a set of grammatical rules where developers write instructions in human-readable text.
- **Formal Notation Languages:** Languages based on mathematical models designed to describe a system rigorously with mathematical precision.
- **Visual Programming Languages:** Languages that use graphical elements to represent actions, variables, and control flow.

- **Phased Integration ("Big Bang"):** Delaying the integration of component software parts until all parts intended for release in a version are complete.
- **Incremental Integration:** Writing and testing a program in small pieces and then combining the pieces one by one.

Static Analysis & Types

- **Type:** A set of values, along with operations that can be performed on those values.
- **Static Checking:** Checking for bugs at compile time (before the program runs).
- **Static Typing:** A kind of static checking where the types of all variables are known at compile time,.
- **Dynamic Checking:** Finding bugs automatically when the code is executed.
- **Static Analysis:** Analyzing code without executing it, generally used to find bugs or ensure conformance to coding standards.
- **FindBugs:** A static analysis tool that examines class or JAR files looking for potential problems by matching bytecodes against a list of bug patterns.
- **Checkstyle:** A static analysis tool that focuses on Java coding style and standards.
- **Arrays:** Fixed-length sequences of another type.
- **Lists:** Variable-length sequences of another type.
- **Immutable Type:** A type whose object value cannot be changed after it is created (e.g., String).
- **Mutable Type:** An object that can be changed after it is created (e.g., StringBuilder, ArrayList).
- **Snapshot Diagrams:** A tool that represents the internal state of a program at runtime, showing each variable with an arrow pointing to its value.
- **Mutating:** Changing the contents of a mutable value by changing references inside that value.
- **Immutable References:** Variables that are assigned once and never reassigned (declared with the keyword final).
- **Iterator:** A mutable object that steps through a collection of elements and returns the elements one by one.

Code Review & Quality

- **Code Review:** Careful, systematic study of source code by people who are not the original author of the code.
- **Coding Style Standards:** Guidelines for writing computer code, prescribing specific formatting, naming conventions, and documentation.
- **DRY (Don't Repeat Yourself):** A principle rooted in the reduction of repetition by encapsulating logic used in more than one place within reusable components.
- **Fail Fast:** A principle where code should reveal its bugs as early as possible (closer to the cause).
- **Magic Numbers:** Constants that appear in code without explanation (other than 0 and 1).
- **Global Variable:** A variable accessible and changeable from anywhere in the program.

Testing & Verification

- **Validation:** The process to uncover problems in a program and increase confidence in its correctness.
- **Verification:** Formal reasoning about a program to construct a formal proof that it is correct.
- **Testing:** Running the program on carefully selected inputs and checking the results.
- **Test-first Programming:** Writing tests before writing any code.
- **Subdomains:** Sets of similar inputs on which the program has similar behavior, used to partition the input space,.
- **Blackbox Testing:** Choosing test cases only from the specification, not the implementation.
- **Whitebox Testing (Glass Box):** Choosing test cases with knowledge of how the function is actually implemented.
- **Coverage:** A measure of how thoroughly a test suite exercises the program (includes Statement, Branch, and Path coverage).
- **Unit Test:** A test that tests an individual module (method or class) in isolation.
- **Integration Test:** A test that tests a combination of modules or the entire program.
- **Automated Testing:** Running tests and checking their results automatically.

- **Regression Testing:** Running all tests after every change to prevent the program from regressing.
- **Regression Test:** A test case added to the suite after fixing a bug to ensure it doesn't come back.

Debugging, Exceptions, & Design

- **Assertion:** A statement that a programmer believes to be true at a specific point in the code, used to catch errors in logic.
- **Modularity:** Dividing up a system into components (modules) that can be designed, implemented, tested, and reused separately.
- **Encapsulation:** Bundling data and corresponding methods together and building walls around a module so it is responsible for its own internal behavior.
- **Scope:** The portion of the program text over which a variable is defined.
- **Exception:** An event occurring during execution that disrupts the normal flow of instructions.
- **Throwing an Exception:** The process where a method creates an exception object containing error information and hands it off to the runtime system.
- **Call Stack:** An ordered list of methods that had been called to get to the method where an error occurred.
- **Exception Handler:** A block of code designed to handle a specific exception.
- **Checked Exceptions:** Exceptions checked at compile time, used to signal special results that a well-written application should anticipate and recover from.
- **Unchecked Exceptions:** Exceptions used to signal bugs, not forced by the compiler to be handled (includes Errors and RuntimeExceptions).
- **Error:** Exceptional conditions external to the application that it usually cannot recover from (e.g., OutOfMemoryError).
- **Runtime Exception:** Exceptional conditions internal to the application usually indicating programming bugs (e.g., NullPointerException).

Specifications

- **Specifications:** A detailed description of what a software component is supposed to do, serving as a contract between implementer and client,.
- **Behavioral Equivalence:** The concept of whether one implementation can be substituted for another based on the client's perspective.

- **Precondition (requires):** An obligation on the client (caller) defining the state in which the method must be invoked,.
- **Postcondition (effects):** An obligation on the implementer defining what the method must do if the precondition holds,.
- **Operational Specifications:** Specifications that give a series of steps the method performs (pseudocode).
- **Declarative Specifications:** Specifications that give properties of the final outcome and its relation to the initial state without detailing intermediate steps.

AI in Software Development

- **Artificial Intelligence (AI):** Technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity, and autonomy.
- **Generative AI (Gen AI):** AI that can create original content (text, images, code, etc.) in response to a user's prompt.
- **Boilerplate Code:** Code that must be written again and again in almost every project with little change.
- **Autocompletion:** AI predicts and suggests the next lines of code.
- **Code Synthesis:** AI automatically writes code based on natural language descriptions.
- **Performance Optimization:** The process of making code run faster, use less memory, and scale better.
- **Threat Detection:** Identifying potential or active attacks targeting an application.
- **Code Auditing:** Systematically reviewing source code to find security flaws or bad practices.
- **SQL Injection:** A security vulnerability where an attacker inserts malicious SQL code into a query via user input.
- **Prompt Driven Development (PDD):** A paradigm where developers use natural language prompts to an LLM to generate necessary code instead of writing it manually.
- **Context (in prompting):** Snippets or files from the codebase provided in a prompt.
- **Specificity (in prompting):** How clearly defined and precise the request is.
- **Scope (in prompting):** How much the LLM is asked to change at once.

- **Prompt Chaining:** Breaking a complex task into smaller, sequential prompts where the output of one feeds into the next.

Machine Learning & Deep Learning

- **Machine Learning:** A field of study that gives computers the ability to learn without being explicitly programmed, or algorithms that detect patterns in data to predict future outcomes,.
- **Supervised Learning:** Learning where training data includes desired outputs (pairs of input and output),.
- **Unsupervised Learning:** Learning where training data does not include desired outputs; the goal is to learn the structure of the data,.
- **Reinforcement Learning:** Learning based on rewards from a sequence of actions,.
- **Hyperparameters:** Parameters that cannot be directly learned from the training process and express "higher-level" properties (e.g., model complexity, learning rate).
- **Artificial Neural Network:** A computational model that mimics the way the human brain operates, processing information at simple elements called neurons.
- **Deep Learning:** A subset of machine learning involving multiple layers of nonlinear processing units and the learning of hierarchical feature representations.
- **Convolutional Neural Networks (CNNs):** Multilayer feedforward networks that use minimal preprocessing and adaptively learn spatial hierarchies of features, commonly used for image recognition.
- **Convolution Layer:** A layer in a CNN that performs convolution operations using filters to scan the input.
- **Pooling Layer:** A down-sampling operation in a CNN that reduces the dimensionality of feature maps.
- **Recurrent Neural Networks (RNNs):** Networks with feedback loops that maintain a hidden state acting as memory, allowing them to capture temporal dependencies.
- **Long Short-Term Memory (LSTM):** A special kind of RNN capable of learning long-term dependencies using gates to control information flow.
- **Transformer Networks:** Models that process input data using self-attention mechanisms, allowing for parallelization and handling long-range dependencies.
- **Generative Adversarial Networks (GANs):** A setup where two neural networks (Generator and Discriminator) are trained in a competitive setting to generate new synthetic data.